

	<pre> ('Bob Jones', 'bob@example.com'); -- Add products INSERT INTO products (product_name, price) VALUES ('Wireless Mouse', 25.00), ('Mechanical Keyboard', 85.50), ('Gaming Monitor', 299.99); -- Simulate orders (Alice buys a mouse, Bob buys a keyboard and a monitor) INSERT INTO orders (user_id, product_id) VALUES (1, 1), -- Alice (user_id 1) buys Mouse (product_id 1) (2, 2), -- Bob (user_id 2) buys Keyboard (product_id 2) (2, 3); -- Bob (user_id 2) buys Monitor (product_id 3) </pre>
Step2:	Fetch the Combined Data (The JOIN Query) <pre> SELECT orders.id AS order_number, users.name AS customer_name, products.product_name, products.price, orders.order_date FROM orders INNER JOIN users ON orders.user_id = users.id INNER JOIN products ON orders.product_id = products.id; </pre>
Step3:	Test the Schema Constraints Test A: The Foreign Key Restriction (ON DELETE RESTRICT) Try to delete the "Wireless Mouse" from the products table while Alice's order exists: <pre>DELETE FROM products WHERE id = 1;</pre> Result: MySQL will block this action and show an error (#1451 - Cannot delete or update a parent row: a foreign key constraint fails). This protects your sales history. Test B: The Cascading Delete (ON DELETE CASCADE) Try to delete the user "Alice Smith" entirely from the system: <pre>DELETE FROM users WHERE id = 1;</pre>

	Result: MySQL allows the deletion. If you run the JOIN query from Step 2 again, Alice's order (Order #1) will be automatically deleted as well. This prevents orphaned data.
	<u>Case Study 2: Indexing</u>
Step1:	<pre>[inventory Table] +-----+ id (PK) -> Automatically indexed (Primary Key) sku (UQ) -> Automatically indexed (Unique Constraint) product_name -> Explicitly Indexed category_unindexed -> Deliberately Unindexed stock_count +-----+</pre>
Step2:	Build the Schema from Scratch <pre>CREATE TABLE inventory (id INT AUTO_INCREMENT PRIMARY KEY, sku VARCHAR(50) NOT NULL UNIQUE, product_name VARCHAR(150) NOT NULL, category_unindexed VARCHAR(100) NOT NULL, stock_count INT NOT NULL, -- Explicitly adding a secondary index to the product name column INDEX idx_product_name (product_name));</pre>
Step3:	Populate Mock Data for Simulation <pre>INSERT INTO inventory (sku, product_name, category_unindexed, stock_count) VALUES ('SKU-001', 'Logitech Wireless Mouse M325', 'Electronics', 150), ('SKU-002', 'Razer DeathAdder Gaming Mouse', 'Electronics', 85), ('SKU-003', 'Dell UltraSharp 27 Inch Monitor', 'Computers', 40), ('SKU-004', 'Apple MacBook Pro 16', 'Computers', 25), ('SKU-005', 'Samsung Galaxy S24 Ultra', 'Smartphones', 60), ('SKU-006', 'Anker PowerCore 20K Power Bank', 'Accessories', 200);</pre>
Step4:	Understanding index Scenario A: Searching the Unindexed Column (Slow Scan) <pre>EXPLAIN SELECT * FROM inventory WHERE category_unindexed = 'Computers';</pre> Look at the Results Panel:

	• type: ALL (This means MySQL had to perform a Full Table Scan). • possible_keys: NULL (MySQL had zero map pathways to follow). • rows: 6 (MySQL physically picked up and checked every single row in the database). Scenario B: Searching the Indexed Column (Instant Search) EXPLAIN SELECT * FROM inventory WHERE product_name = 'Apple MacBook Pro 16'; Look at the Results Panel: • type: ref (This means it used a non-unique index lookup). • key: idx_product_name (MySQL actively utilized our custom index map). • rows: 1 (MySQL went straight to the targeted row without reading anything else).
	Part 1: Creating a Composite Index (Multi-Column) ALTER TABLE inventory ADD INDEX idx_name_stock (product_name, stock_count); Step 2: Analyze the Optimization EXPLAIN SELECT * FROM inventory WHERE product_name = 'Razer DeathAdder Gaming Mouse' AND stock_count > 0; • Why it works perfectly: Instead of using one index to find the product name and then scanning the table to check the stock, MySQL uses the composite index <code>idx_name_stock</code> to satisfy both requirements in a single lookup pass.